Micro Project Proposal

IoT-Based Weather Monitoring System

Vivek A , Roll No: 26
Avigna Suraj Nambiar , Roll No: 10
S4 EV, Department of Electronics and Communication

1 Introduction

The rapid advancement of environmental sensing technologies has emphasized the need for intelligent solutions to monitor atmospheric conditions in real-time. Traditional manual methods of recording temperature, humidity, and pressure are not only labor-intensive but also prone to delayed response, resulting in inaccurate data collection and poor environmental management. To overcome these limitations, the deployment of automated IoT-based monitoring nodes has become essential.

This project proposes the design and implementation of a compact IoT-based Weather Monitoring System using the ESP32 NodeMCU microcontroller. The system integrates high-precision sensors—the DHT22 for temperature and humidity, and the BMP280 for atmospheric pressure. Sensor data are transmitted via Wi-Fi and hosted on a built-in web server accessible from any device on the same network, providing a live real-time dashboard. By leveraging IoT technology, the system enables continuous, remote environmental monitoring, providing a practical and scalable solution for smart-city and agricultural applications.

2 System Architecture

The system integrates temperature and humidity sensing (DHT22) and barometric pressure sensing (BMP280) with the ESP32 NodeMCU, which acts as the central controller. The ESP32 connects via Wi-Fi and hosts a local web server that any browser on the same network can access for real-time monitoring. The DHT22 communicates over a single digital wire (GPIO4), while the BMP280 uses the I2C bus (SDA on GPIO21, SCL on GPIO22).

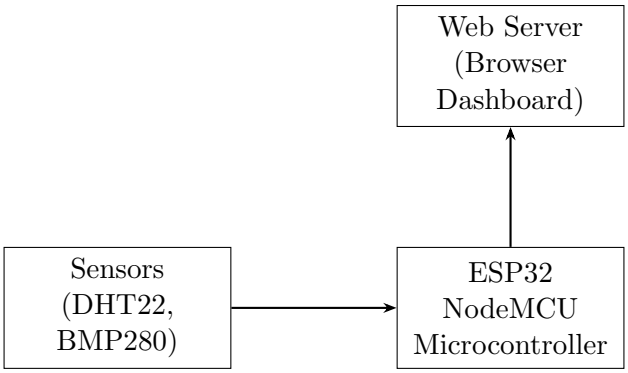


Figure 1: Block Diagram of IoT Weather Monitoring System

3 Circuit Diagram

The following circuit shows the connection of ESP32 NodeMCU with DHT22 sensor and BMP280 barometric pressure sensor.

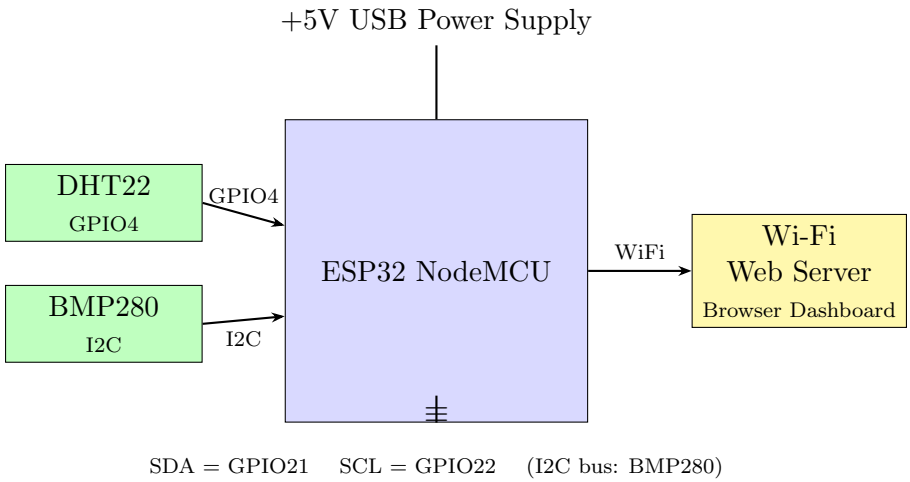


Figure 2: Circuit Diagram of IoT Weather Monitoring System

4 Components

- **ESP32 NodeMCU:** A powerful dual-core microcontroller with integrated Wi-Fi and Bluetooth, serving as the system's brain and web host.
- **DHT22 Sensor:** A high-precision digital sensor providing calibrated temperature and humidity readings via a single-bus protocol.
- **BMP280 Sensor:** An absolute barometric pressure sensor that enables weather forecasting and altitude estimation through I2C communication.

- **Power Supply:** A regulated 5 V micro-USB input, which is internally regulated to 3.3 V by the NodeMCU for sensor operation.
- **Interconnects:** High-quality jumper wires and a standard breadboard for reliable signal routing.

5 Functional Explanation

The IoT Weather Monitoring System primarily relies on accurate sensing and timely communication to log and display environmental conditions.

Sensors

- **DHT22 Sensor:** Measures ambient temperature and relative humidity. It communicates via a single-wire digital protocol on GPIO4. The DHT22 offers high accuracy (± 0.5°C, $\pm 2\%$ RH) and a wide measurement range.
- **BMP280 Sensor:** Measures atmospheric pressure and can derive approximate altitude. It uses piezo-resistive technology and communicates over the I2C bus (SDA = GPIO21, SCL = GPIO22).

Controller

The **ESP32 NodeMCU** acts as the main processing unit and IoT gateway. It performs data acquisition, hosts an HTTP web server, and serves a real-time dashboard accessible via any browser on the same network.

Web Dashboard

The dashboard UI is served entirely from the ESP32's flash memory (`PROGMEM`). It features a dark-mode interface with sensor cards for the DHT22 and BMP280. JavaScript running in the browser silently fetches `/data` every 2.5 seconds using the `fetch()` API to update values dynamically.

6 Project Methodology

Sensor data acquisition occurs at regular intervals using a non-blocking `millis()` timer, ensuring the web server remains responsive to client requests at all times. When a browser connects to the root endpoint, the ESP32 serves the full HTML dashboard. The browser then autonomously polls the `/data` endpoint every 2.5 seconds, receiving a JSON response that JavaScript parses and renders on-screen. This asynchronous method ensures high performance and prevents system hangs.

7 Arduino Code Description

The firmware for this project is developed using the Arduino framework in C++. It follows an asynchronous, non-blocking execution model to ensure high availability for the web server while maintaining accurate sensor sampling.

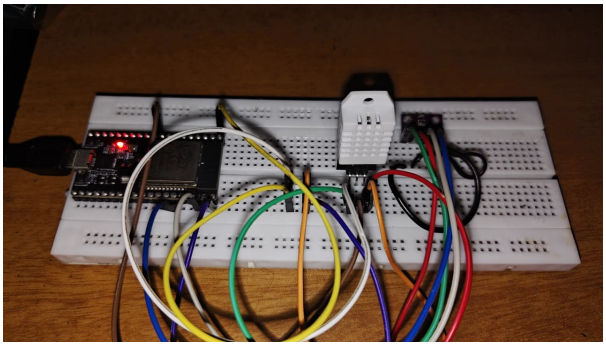
- **Library Integration:** The code utilizes the `Adafruit_BMP280` and `DHT` libraries for low-level sensor communication. The `WiFi.h` and `WebServer.h` libraries handle the networking stack.
- **Networking:** During the `setup()` phase, the ESP32 initializes a connection to the specified local Wi-Fi access point. It then configures the I2C and single-wire interfaces for the sensors.
- **Web Handling:** Two primary routes are defined: `handleRoot()`, which serves the HTML dashboard, and `handleData()`, which returns the latest sensor values in JSON format.
- **Execution Loop:** The `loop()` function continuously processes incoming HTTP requests. A `millis()`-based timer triggers sensor readings every 2.5 seconds to ensure the dashboard remains fluid and responsive.

8 Complete Pin Interfacing Logic

Component	**Component Pin**	**ESP32 Pin**	**Protocol**
DHT22	VCC	3.3 V	—
DHT22	GND	GND	—
DHT22	DATA	GPIO26	1-Wire Digital
BMP280	VCC	3.3 V	—
BMP280	GND	GND	—
BMP280	SDA	GPIO21	I2C
BMP280	SCL	GPIO22	I2C
Power Supply	5 V USB	VIN	—

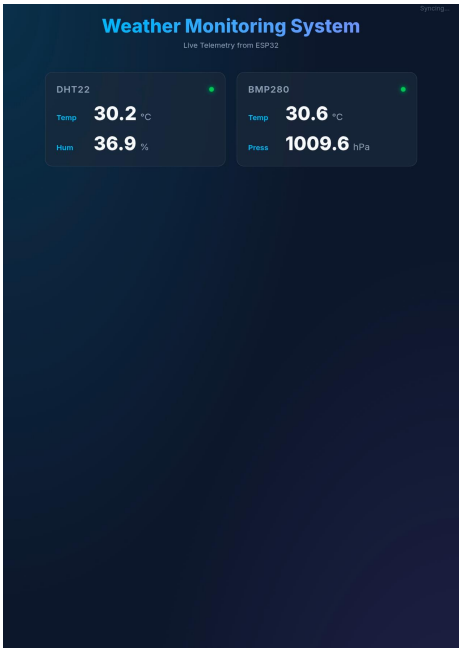
Table 1: Complete Pin Interfacing Logic

9 Image of Working Model



(a) Detailed hardware circuit on breadboard

(b) Integrated IoT system on model house



(c) Live Weather Monitoring System web dashboard interface

Figure 3: Working Model and Interface of the IoT Weather Monitoring System

10 Expected Outcomes

- **Real-Time Monitoring:** Environmental parameter monitoring accessible from any device on the local network via a standard browser.
- **Dynamic Visualization:** Accurate and responsive display of temperature, humidity, and atmospheric pressure on a professional dashboard.
- **Local Hosting:** Reliable data delivery via the ESP32's built-in web server, ensuring privacy and high speed by avoiding external cloud dependencies.
- **System Stability:** A robust firmware implementation that remains responsive over long periods of continuous operation.

11 Future Scope

The current architecture provides a strong foundation for various enhancements:

- **Data Logging:** Integration of an SD card module or SPIFFS storage to log historical sensor data for long-term trend analysis.
- **Deep Sleep Mode:** Utilizing the ESP32's low-power modes for battery-operated deployment in remote agricultural sites.
- **Mobile Application:** Developing a dedicated Flutter or React Native app to provide push notifications when weather thresholds are exceeded.

12 Conclusion

The proposed IoT-based Weather Monitoring System offers an efficient and cost-effective solution for continuous environmental sensing. By leveraging the ESP32 NodeMCU's processing power and Wi-Fi capabilities, the system provides a seamless bridge between physical sensor data and a digital user interface. The DHT22 and BMP280 sensors ensure high-resolution readings, making the system practical for standalone deployment or integration into larger smart-city infrastructures.

References

1. K. Ashton, "That 'Internet of Things' Thing," *RFID Journal*, 2009.
2. P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, Protocols, and Applications," *Journal of Electrical and Communication Engineering*, 2017.
3. Espressif Systems, *ESP32 Series Datasheet*, Rev. 3.5, 2023.
4. Bosch Sensortec, *BMP280 Digital Pressure Sensor Data Sheet*, Rev. 1.27, 2018.